R3.04 – Qualité de développement
Les principes SOLID

Pierre Pomeret-Coquot
pierre.pomeret@iut-rodez.fr
Université Toulouse Capitole – IUT de Rodez

2026 – 2027







Plan du cours

1. Rappels
   - Les principes SOLID
   - Refactoring - quelques lignes directrices

2. Application `tp_solid`
   - Architecture de l'application
   - Identification de problèmes de maucaise conception

CC BY

Les principes SOLID

- Single responsability principle
  → Une classe = une responsabilité
- Open/closed principle
  → Ouvert aux extensions / fermé aux modifications
- Liskov's substitution principle
  → On hérite du contrat de la classe mère
- Interface segregation principle
  → Interfaces implémentables et implémentées
- Dependency inversion principle
  → Dépendences les plus faibles possibles

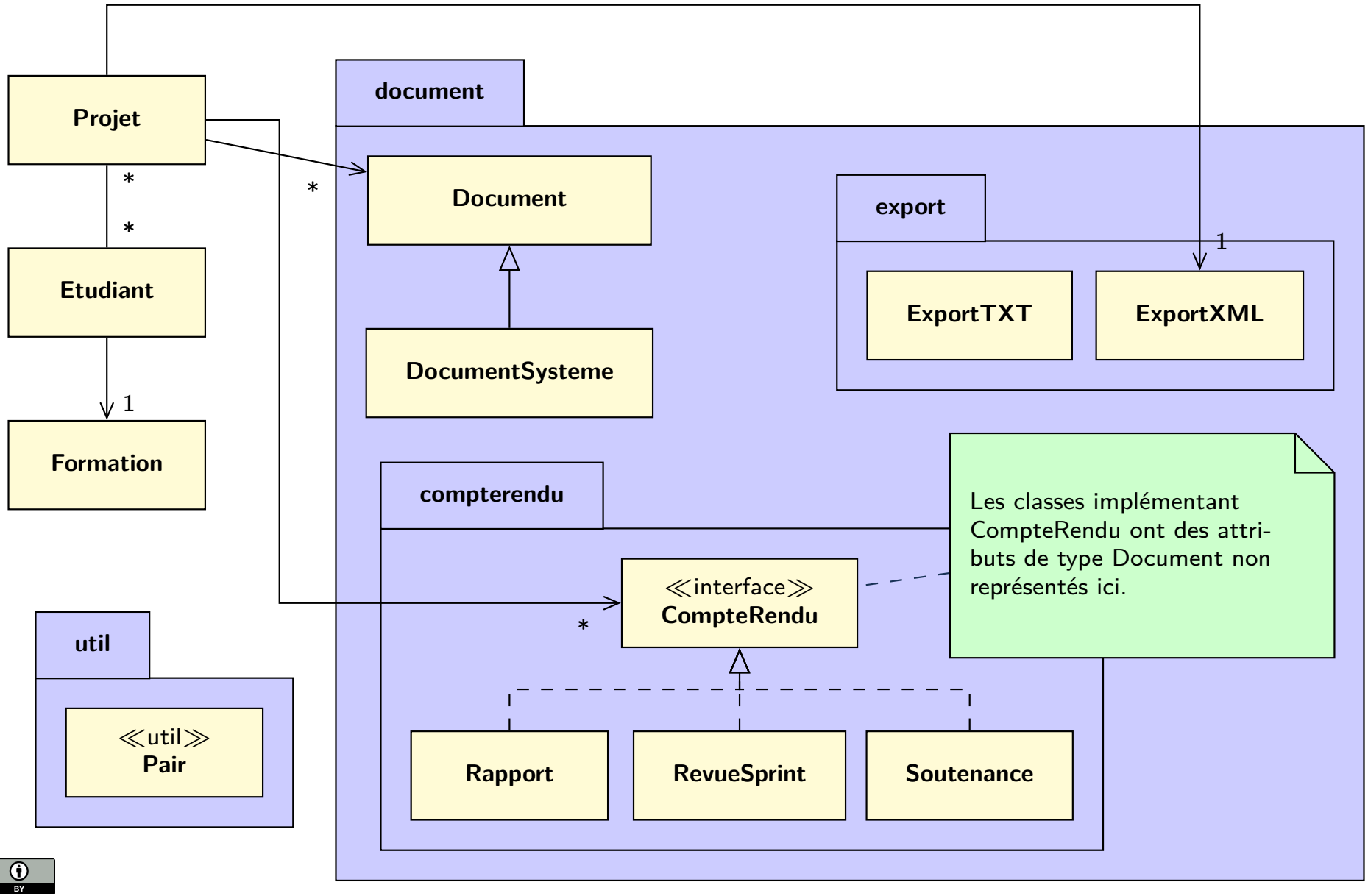
CC BY

Refactoring – quelques lignes directrices

- Single responsability principle
  - Ne pas mettre tout le code dans une classe...
  - \+ Mais diviser en plusieurs classes dédiées.
- Open/closed principle
  - Ne pas gérer la variété des comportements avec des if...then...
  - \+ Mais utiliser l'héritage.
- Liskov's substitution principle
  - Ne pas utiliser l'héritage lorsque les contrats sont différents...
  - \+ Mais utiliser la composition.
- Interface segregation principle
  - Ne pas inclure dans l'interface des méthodes non-implémentables...
  - \+ Mais diviser en plusieurs interfaces (en cascade).
- Dependency inversion principle
  - Ne pas faire dépendre de classes concrètes...
  - \+ Mais faire dépendre de classes abstraites et interfaces.

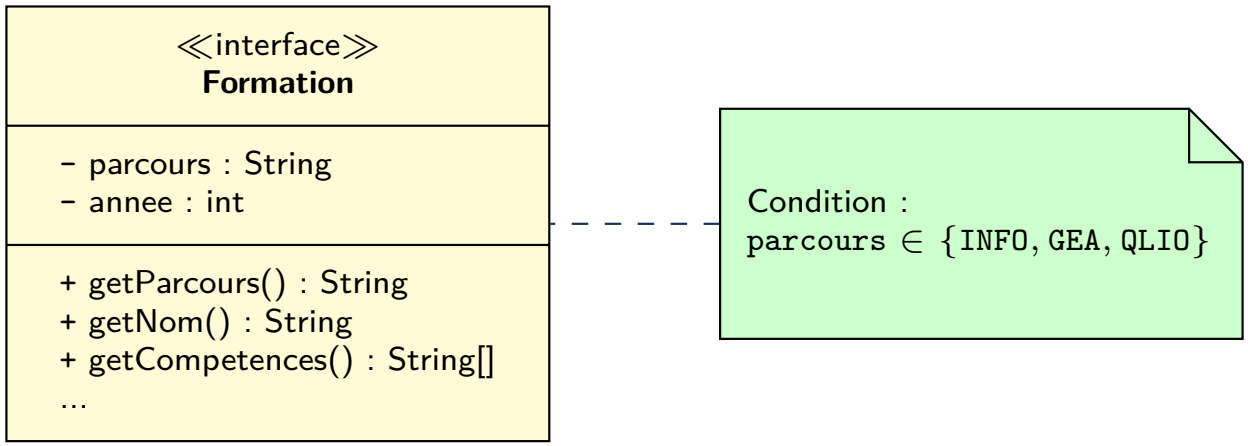
CC BY

Diagramme général



CC BY

Classe Formation




```java
public class Formation {

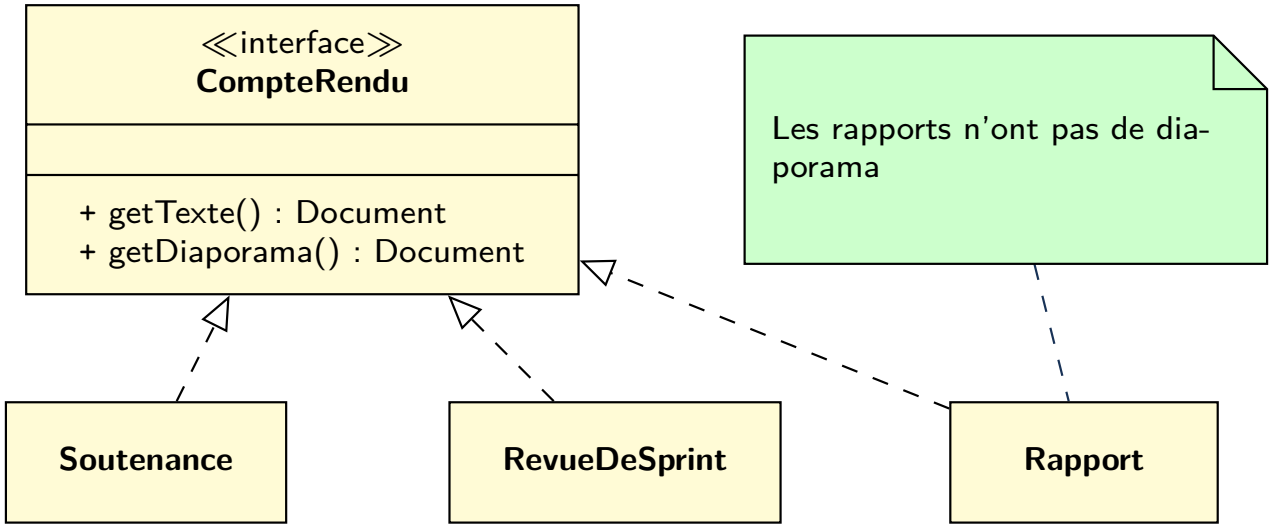
  private int annee;
  private String parcours;

  public Formation(String p, int a) {
    this.parcours = p;
    this.annee = a;
  }

  public String getNom() {
    String nom;
    switch (this.parcours) {
      case "INFO" :
        nom = "Informatique";
        break;
      case "GEA":
        nom = "Gestion des Entrep...";
        break;
      case "QLIO":
        nom = "Qualité, Logistique...";
        break;
      default:
        throw new
              IllegalArgumentException("
              Parcours ne peut pas être
              '" + parcours + "'");
    }
    return nom;
  }
}
```

CC BY

Interface CompteRendu




```java
public interface CompteRendu {
  Document getTexte();
  Document getDiaporama();
}
```

```java
public class Soutenance implements
      CompteRendu {
  /* ... */
  public Document getDiaporama() {
    return diaporama;
  }
}
```

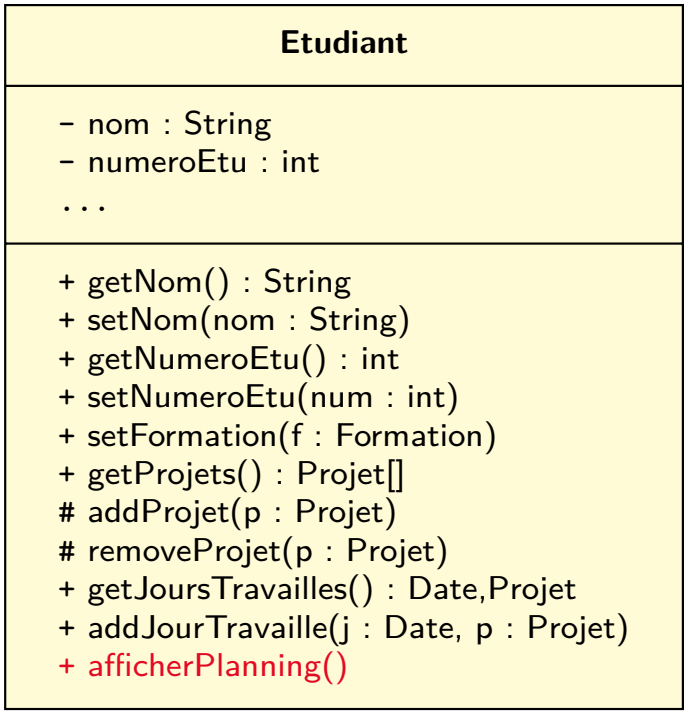
```java
public class Rapport implements
      CompteRendu {

  /* ... */

  public Document getDiaporama() {
    throw new
          UnsupportedOperationException
          ("Les rapports n'ont pas de
          diaporama");
  }
}
```

CC BY

Classe Etudiant

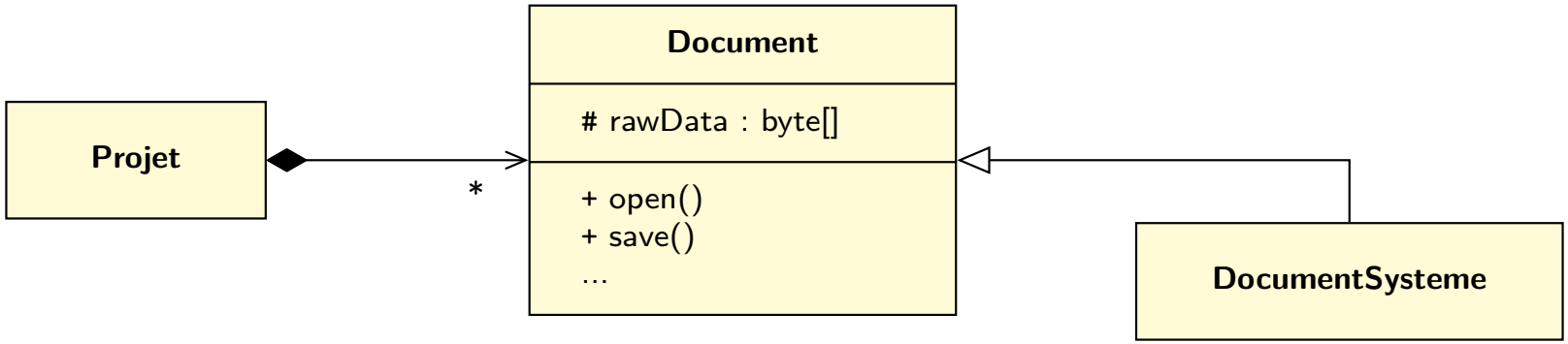



```java
public void afficherPlanning() {
  SimpleDateFormat df = new SimpleDateFormat("dd/MM/yyyy");
  String nomFormation;
  switch (this.getFormation().getParcours()) {
  case "INFO" :
    nomFormation = "Informatique";
    break;
  case "GEA":
    nomFormation = "Gestion des Entreprises...";
    break;
  case "QLIO":
    nomFormation = "Qualité, Logistique...";
    break;
  default:
    nomFormation = "Formation inconnue'";
  }

  System.out.println("Planning de" + this.getNom() + " (" + nomFormation + ")");
  System.out.println("--------");
  for (Pair<Date,Projet> p : joursTravailles) {
    Date date = p.getL();
    Projet projet = p.getR();
    System.out.println("+ " + projet.getNom() + " (le " + df.format(date) + ")");
  }
}
```

CC BY

Classe DocumentSysteme

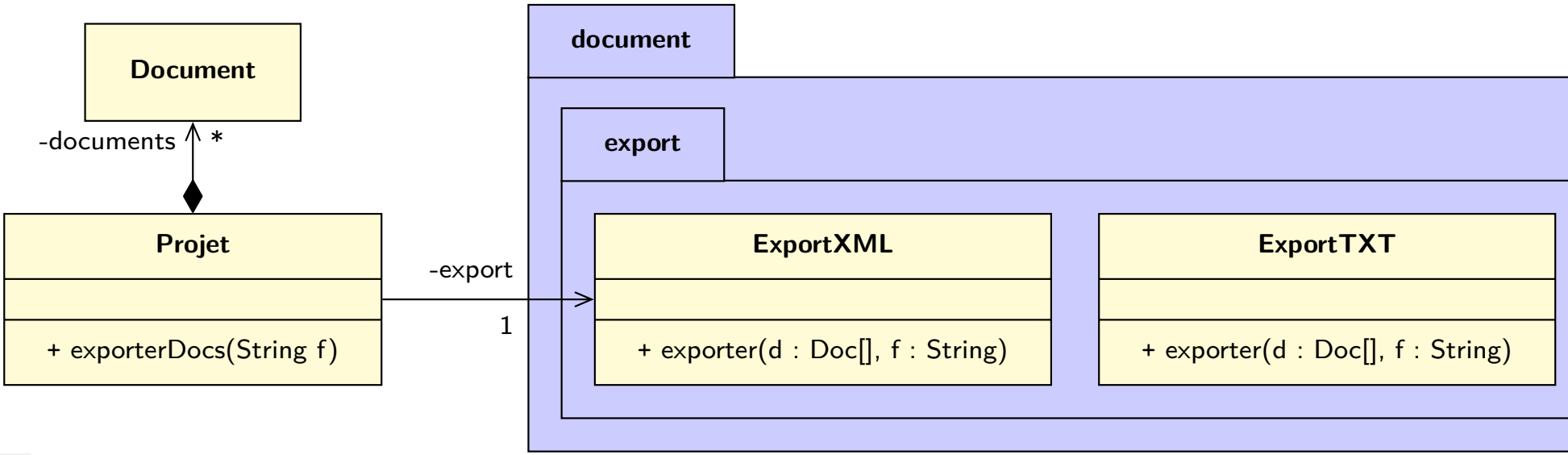



```java
public class Document {
  byte[] rawData
  public void open() throws IOException {
    rawData = Files.readAllBytes(Paths.get(filename));
  }
  public void save() throws IOException {
    Files.write(Paths.get(filename), rawData);
  }
}
```

```java
public class DocumentSysteme extends Document {
  @Override
  public void save() throws IOException {
    String alternative_filename = filename + ".tmp";
    Files.write(Paths.get(alternative_filename), rawData);
    System.err.println("Warning: Les documents systèmes sont en lecture seule, une
        copie du document a été enregistrée sous '" + alternative_filename + "'.");
  }
}
```

CC BY

Paquetage document.export




```java
public class Projet {
  private List<Document> documents;
  private ExportXML export = new ExportXML();
  public void exporterDocuments(String filename) throws IOException {
    this.export.exporter(this.documents, filename);
} }
```

```java
public class ExportXML {
  public void exporter(List<Document> docs, String filename) throws IOException {
    Document docExport = new Document(filename, "txt");
    String contenu = "<?xml version=\"" + xmlVersion + "\"?>\n<documents>\n";
    for (Document doc : documents) {
      doc.open();  contenu = contenu +  "\t<document> [...] </document>\n";
    }
    contenu = contenu + "</documents>";
    docExport.setRawData(contenu.getBytes());
    docExport.save();
} }
```